

Network and Cyber Security

Bachelor in Computer Engineering

Er. Anuj Sherchan
Assistant Professor

Unit 6: Web Application Security: Standards and Practices

- Outline:
- 6.1 OWASP Top 10
- 6.2 Ways to maintain Security
 - 6.2.1 Authentication and Authorization
 - 6.2.2 Data Protection in Storage and Transit, Data Validation and Parameter Validation
 - 6.2.3 Error handling and Exception Management
 - 6.2.4 User and Session Management
 - 6.2.5 Auditing and Logging
- 6.3 Security testing
 - 6.3.1 Broken Access Control
 - 6.3.2 Various failures-Cryptographic Failures, Identification and Authentication Failures, Software and Data Integrity Failures, Security Login and Monitoring Failures

Web Application Security

- **Web application security**, often termed Web AppSec, refers to the discipline of protecting web-based applications from malicious threats and inherent vulnerabilities.
- It involves a suite of practices, tools, and methodologies aimed at ensuring that these applications operate securely while guarding sensitive data against unauthorized access or manipulation.
- Distinct from broader **network security**, which secures infrastructure, web application security zeroes in on the application layer.
- It tackles issues like flawed code, misconfigured settings, and logical errors.
- Web application security covers the full software development lifecycle (SDLC).
- It includes secure coding, regular testing, and continuous monitoring from design through deployment.
- A technical nuance worth noting: vulnerabilities often stem from overlooked edge cases in code, such as unvalidated API inputs, which experts prioritize mitigating early.

Web Application Security

- **How Does Web Application Security Work?**
- Web application security functions through a layered strategy, blending preventive and responsive tactics to counter diverse web application security threats.
- Its operational framework can be distilled into key components:
- **Proactive defenses**
- Secure development practices form the bedrock.
- Developers employ techniques like input sanitization to thwart injection attacks and encode outputs to prevent **cross-site scripting**(XSS), which can compromise **browser security** if left unchecked.
- Web application security testing, including static (SAST) and **dynamic (DAST) methods**, identifies flaws pre-deployment.
- Regular patching of software components addresses known exploits swiftly.

Web Application Security

- **Reactive safeguards**
- Once live, applications benefit from tools like Web application firewalls (WAFs), which filter malicious traffic using rule-based detection.
- **Intrusion detection systems**(IDS) monitor for anomalies, while well-defined incident response plans limit breach impact.
- WAFs must be tuned to an application's unique traffic patterns to avoid false positives, a step often skipped by novices.
- **Ongoing vigilance**
- Regular monitoring watches user activity and system logs, allowing threats to be spotted in real time.
- Following web application security standards keeps systems aligned with industry guidelines like those from OWASP.
- These combined approaches create strong protection against changing threats.

Web Application Security

4 Key Benefits of Web Application Security



Web Application Security

- **Data protection:** Comprehensive security measures safeguard sensitive information, including customer data, intellectual property, and business intelligence.
- **Regulatory compliance:** Robust security practices facilitate compliance with regulatory frameworks, including GDPR, HIPAA, **PCI DSS**, and SOC2.
- Non-compliance penalties can reach substantial figures. **GDPR violations**, for instance, can result in fines up to €20 million or 4% of global annual revenue.
- **Business continuity:** Security incidents frequently disrupt operations, leading to downtime and revenue loss.
- Secure applications maintain availability even during attack attempts, preserving business operations and revenue streams.
- According to **ITIC's 2024 hourly cost of downtime survey**, 90% of firms report hourly downtime costs exceeding \$300,000, with 41% of enterprises experiencing costs between \$1 million and over \$5 million per hour.

Web Application Security

- **Reputation protection:** Security breaches invariably damage brand reputation and erode customer trust.
- Customers usually avoid companies that experience **data breaches**.
- Strong security measures show that an organization is committed to protecting everyone's interests.
- **Cost efficiency:** Addressing security proactively proves significantly more economical than reactive measures.
- Hacker One indicates that security vulnerabilities **cost 30 times more to fix** during production than during development.
- Early detection and remediation prevent costly emergency patching, incident response, and breach notification expenses.

Web Application Security

- **Competitive advantage:** Organizations with demonstrable security capabilities increasingly gain marketplace advantages.
- Enterprise customers increasingly evaluate vendor security postures during procurement, making robust security a business differentiator.

OWASP Top 10

- The Open Worldwide Application Security Project (OWASP) is an open community dedicated to enabling organizations to develop, purchase, and maintain applications and APIs that can be trusted.
- OWASP is a different kind of organization and provides freedom from commercial pressures allows us to provide unbiased, practical, and cost-effective information about application security.
- OWASP is not affiliated with any technology company, although they support the informed use of commercial security technology.
- OWASP produces many types of materials in a collaborative, transparent, and open way.
- The OWASP Foundation is the non-profit entity that ensures the project's long-term success.

OWASP Top 10

- Almost everyone associated with OWASP is a volunteer, including the OWASP board, chapter leaders, project leaders, and project members.
- It support innovative security research with grants and infrastructure.
- The OWASP Top Ten 2025 has been officially released, reflecting significant shifts in software security risks due to evolving development practices and threat landscapes.
- The 2025 edition introduces two new categories and consolidates others to address systemic weaknesses across the software development lifecycle.

OWASP Top 10

- The updated list includes:
- **A01:2025 Broken Access Control:** Still the leading risk, covering flaws that allow attackers to bypass authorization or gain unauthorized access.
- **A02:2025 Security Misconfiguration:** Addresses weak default settings, exposed services, and inconsistent security controls across environments.
- **A03:2025 Software Supply Chain Failures:** Expanded to include vulnerabilities in dependencies, CI/CD systems, and distribution infrastructure, with a focus on malware in software ecosystems, compromised maintainers, and tampered build processes.

OWASP Top 10

- **A04:2025 Cryptographic Failures:** Involves insecure or outdated encryption practices that expose sensitive data.
- **A05:2025 Injection:** Classic input-validation flaws such as SQL, OS, and template injection remain common across modern stacks.
- **A06:2025 Insecure Design:** Focuses on risks introduced by poor architectural decisions or lack of threat modeling.
- **A07:2025 Authentication Failures:** Covers issues in login flows, weak password policies, or session handling that lead to unauthorized access.

OWASP Top 10

- **A08:2025 Software or Data Integrity Failures:** Addresses flaws where code or data can be modified or tampered with, particularly in update mechanisms or pipelines.
- **A09:2025 Logging & Alerting Failures:** Highlights gaps in monitoring or alerting that allow attacks to go undetected.
- **A10:2025 Mishandling of Exceptional Conditions:** A new category for 2025, focusing on unsafe error handling and system resilience when failures occur, emphasizing the need for secure and predictable failure modes.

Ways to maintain Security

- **Authentication and Authorization**
- Authentication and Authorization are two fundamental security processes that work together to control access to systems and data.
- **Authentication:** Confirms the user's identity (proves who the user is).
- **Authorization:** Controls what the verified user is allowed to do (decides what they can access).
- **Order:** Authentication always happens first, and authorization follows afterward.

Ways to maintain Security

Working of Authentication and Authorization

Authentication



Confirms users are who they say they are.

Authorization

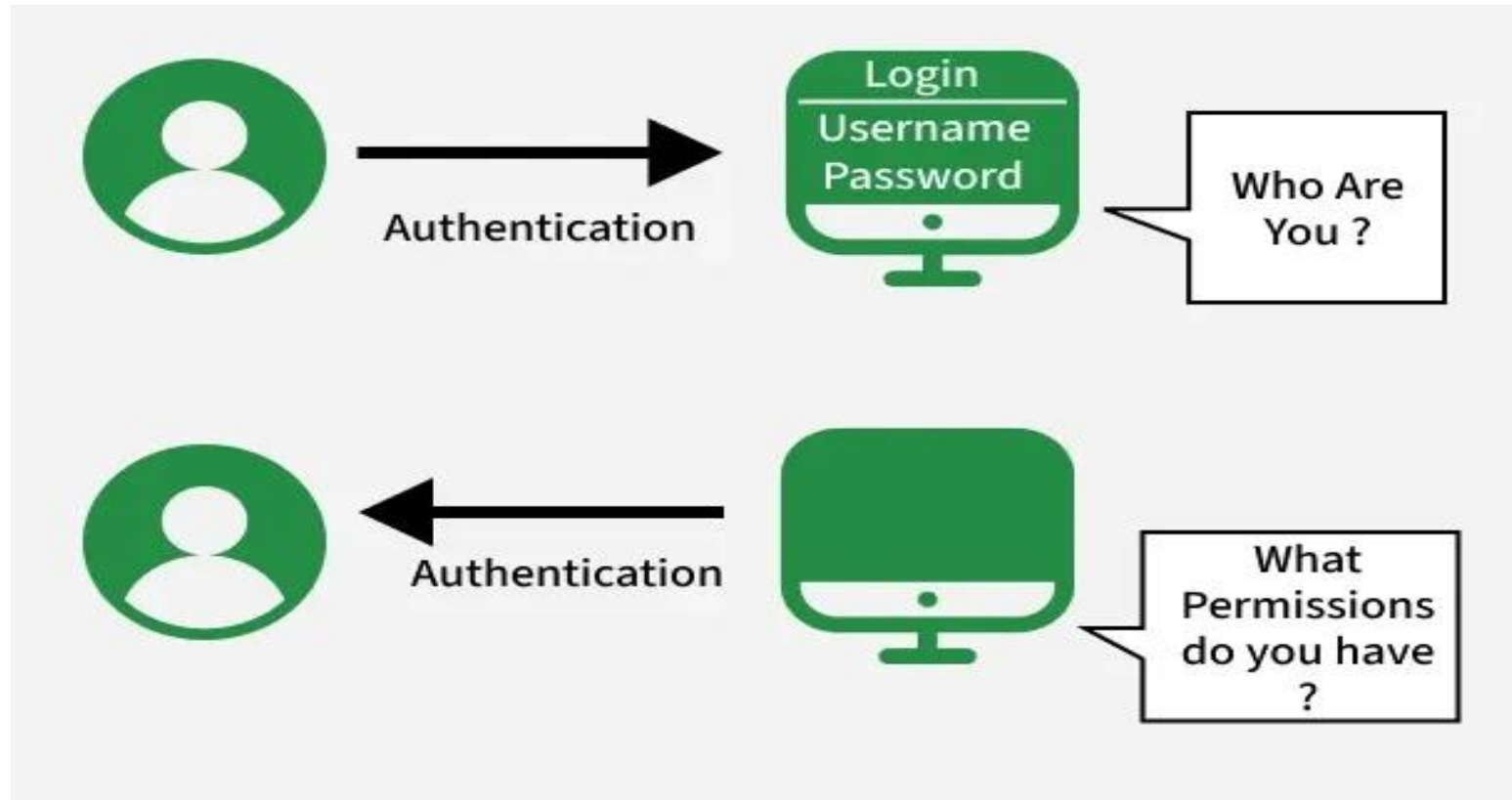


Gives users permission to access a resource

Ways to maintain Security

- **Authentication**
- Authentication is the process of verifying the identity of a user or system.
- It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.
- **How it works:**
- User enters credentials (password, OTP, biometrics)
- System verifies the credentials
- If valid, the user is successfully authenticated

Ways to maintain Security



Ways to maintain Security

- **Authorization**
- Authorization determines the access rights and permissions of an authenticated user.
- It decides what resources the user can access and what actions they are allowed to perform.
- **How it works:**
- System checks the user's roles or permissions
- Grants or denies access to resources
- Ensures the user can perform only allowed actions

Ways to maintain Security

Authentication	Authorization
Verifies who the user is	Determines what the user can access
Performed before authorization	Happens after authentication
Requires login details (username, password, biometrics)	Requires user roles, privileges, or access levels
Determines if the user is valid	Determines what permissions the valid user has
Uses ID Tokens	Uses Access Tokens
Governed by OpenID Connect (OIDC)	Governed by OAuth 2.0
Credentials can be changed by the user	Permissions can only be changed by the system owner
Visible to the user (entering credentials)	Not visible to the user (handled in the background)
Examples: Password, OTP, fingerprint, face recognition	Examples: Admin rights, read/write access, role-based permissions

Ways to maintain Security

- **Data protection in transit, in storage, and in use**
- Securing digital information requires an understanding of how data exists in two fundamental states: **data at rest vs data in transit**.
- “Data at rest” refers to stored information on hard drives, databases, or cloud servers.
- “Data in transit” refers to data that is actively moving across networks or between devices.
- Each state presents distinct security challenges.
- Stored data faces risks such as unauthorized access and insider threats, while data in transit is vulnerable to interception.
- Data encryption plays a critical role in mitigating these risks, ensuring that sensitive information remains protected, whether stationary or in motion.

Ways to maintain Security



Data at Rest

- Hard drives
- Databases
- Cloud storage

Risks:

- Unauthorized access
- Data breaches



Data in Transit

- Emails
- File transfers
- API communications

Risks:

- Interception
- MITM attacks

Ways to maintain Security

- **Data at Rest vs Data in Transit**
- **Data in transit** is actively moving across networks—such as emails, file transfers, or API communications—making it vulnerable to interception.
- In contrast, **data at rest** remains stored on devices, databases, or cloud environments, where unauthorized access is the primary concern.
- **Data in Transit Security**
- **Encryption protocols (TLS, SSL, IPsec):** Cryptographic protocols to secure data transmission by encrypting network traffic to prevent unauthorized access
- **Secure transmission protocols (HTTPS with TLS):** Encrypt data in transit to protect it from interception
- **Authentication mechanisms (MFA, digital certificates):** Verify user identities and ensure that only authorized parties can access transmitted data

Ways to maintain Security

- **Data at Rest Security**
- **Encryption of stored files and databases:** Encrypting stored data ensures that even if unauthorized access occurs, the data remains unreadable without the correct decryption key
- **Access control policies (least privilege, role-based permissions):** Implementing strict access controls limits data exposure by ensuring only authorized users can access sensitive information
- **Physical security measures (restricted server access, hardware encryption):** Securing physical data storage locations and using hardware-based encryption protects data from unauthorized access to hardware

Ways to maintain Security

- **Characteristics and Vulnerabilities of Data States**
- Each data state requires tailored protection strategies.
- Data in transit is particularly vulnerable due to its exposure during transmission, while data at rest is a high-value target for attackers seeking stored information.
- **Data in Transit Risks**
- **Interception attacks:** Hackers can eavesdrop on unencrypted data as it moves across networks
- **Man-in-the-middle attacks:** Attackers insert themselves between sender and receiver to alter or steal data
- **Session hijacking:** Unauthorized access to active communication sessions allows data theft

Ways to maintain Security

- **Data at Rest Risks**
- **Unauthorized access:** Weak access controls enable malicious insiders or attackers to retrieve sensitive data
- **Data breaches:** Poorly secured data storage systems can be compromised, leading to large-scale information leaks
- **Physical security threats:** Stolen or lost devices containing unencrypted data increase exposure risks

Ways to maintain Security

- **Data-in-Transit Encryption**
- Encryption prevents interception and unauthorized modification by securing communication channels.
- Common methods include:
- **TLS (Transport Layer Security) and SSL (Secure Sockets Layer):** Encrypt web traffic and email communications
- **E2EE (End-to-end encryption):** Ensures that only the sender and recipient can access message contents
- **VPNs (Virtual Private Networks):** Encrypts network connections to prevent data exposure

Ways to maintain Security

- **Data-at-Rest Encryption**
- Encryption protects stored information from unauthorized access, even if a device or system is compromised.
- Key techniques include:
- **Full disk encryption:** Encrypts entire data storage devices, rendering stolen data unreadable
- **Database encryption:** Secures sensitive records in structured data storage environments
- **File-level encryption:** Encrypts individual files for granular access control

Ways to maintain Security

Best Practice	Data at Rest	Data in Transit
Encryption	AES-256	TLS, SSL, E2EE
Access Control	Role-based access, least privilege	MFA, digital certificates
Regular Audits	Data classification	Traffic monitoring

Ways to maintain Security

- **Data Validation**
- Data validation serves as a protective barrier against erroneous, malicious, or unexpected data, contributing significantly to the overall robustness and reliability of your web application.
- **Understanding Data Validation: Why It Matters**
- Data validation involves inspecting and verifying user input to ensure that it adheres to predefined constraints and standards.
- This process is crucial for various reasons:
- **Preventing Security Vulnerabilities:** Unvalidated data can be a breeding ground for security vulnerabilities such as SQL injection, cross-site scripting (XSS), and more.
- Data validation acts as an effective countermeasure against these threats.

Ways to maintain Security

- **Maintaining Data Integrity:** Validating data before it enters the system helps maintain data integrity by ensuring that only correct and meaningful information is processed.
- **Enhancing User Experience:** Proper data validation provides immediate feedback to users, preventing frustrating errors and improving the overall user experience.
- **Ensuring Accurate Business Logic:** Incorrect or inconsistent data can lead to flawed business logic and incorrect results.
- Validation helps maintain the accuracy of your application's underlying processes.

Ways to maintain Security

- **Best Practices for Data Validation**
- **Use Server-Side Validation as the Primary Line of Defense:** Rely on server-side validation for security and data integrity. Client-side validation should complement server-side validation, not replace it.
- **Sanitize Data:** Alongside validation, sanitize input data by removing potentially harmful characters or code to prevent injection attacks.
- **Use Framework-Provided Validation Libraries:** Most web frameworks offer built-in validation libraries that simplify and standardize the validation process.
- **Implement Error Handling:** Provide clear error messages that guide users toward valid input. Avoid revealing too much information about the nature of validation errors for security reasons.
- **Regularly Update Validation Rules:** As your application evolves, ensure that validation rules are adjusted to accommodate new features and requirements.

Ways to maintain Security

- **Parameter Validation**
- Parameter validation is a critical component of web application security, serving as a primary defense against injection attacks, cross-site scripting (XSS), and other common vulnerabilities.
- It involves **verifying the type, length, format, and content of all incoming data from potentially untrusted sources, including user inputs, URL parameters, HTTP headers, and API requests.**
- This validation should occur as early as possible in the data flow, ideally at the server-side, since client-side validation can be bypassed by attackers who disable JavaScript or use web proxies.

Ways to maintain Security

- For URL parameters, validation must include **type checking, length limits, and format verification using regular expressions or pattern matching**.
- For instance, a parameter expected to be an integer should be validated as such, and its length should not exceed a defined maximum to prevent buffer overflows or denial-of-service attacks.
- Special attention is required for inputs that will be used in specific contexts, such as database queries or HTML output.

Ways to maintain Security

- **Error Handling and Exception Management**
- Proper Error Handling and Exception Management are critical components of Web Application Security.
- Poorly managed errors can expose sensitive system details, aid attackers in reconnaissance, and increase the attack surface.
- **Key Security Risks of Improper Error Handling**
- **Information leakage:** Stack traces, database errors, or server details (e.g., OS, framework versions) can reveal exploitable vulnerabilities.
- **Authentication/authorization enumeration:** Specific error messages (e.g., "Invalid password" vs. "User not found") allow attackers to determine valid usernames.
- **Denial of Service (DoS):** Unhandled exceptions may consume system resources, leading to service degradation or crashes.
- **Logging vulnerabilities:** Sensitive data (e.g., passwords, tokens) logged in plaintext can be exploited if logs are compromised.

Ways to maintain Security

- **Error Handling and Exception Management**
- Exception handling is a programming concept that allows an application to respond to different error states (like network down, or database connection failed, etc) in various ways.
- Handling exceptions and errors correctly is critical to making your code reliable and secure.
- Error and exception handling occurs in all areas of an application including critical business logic as well as security features and framework code.
- Error handling is also important from an intrusion detection perspective.
- Certain attacks against your application may trigger errors which can help detect attacks in progress.

Ways to maintain Security

- **Error Handling Mistakes**
- Researchers at the University of Toronto have found that even small mistakes in error handling or forgetting to handle errors can lead to catastrophic failures in distributed systems.
- Mistakes in error handling can lead to different kinds of security vulnerabilities.
- **Information leakage:** Leaking sensitive information in error messages can unintentionally help attackers.
- For example, an error that returns a stack trace or other internal error details can provide an attacker with information about your environment.
- Even small differences in handling different error conditions (e.g., returning "invalid user" or "invalid password" on authentication errors) can provide valuable clues to attackers.
- As described above, be sure to log error details for forensics and debugging purposes, but don't expose this information, especially to an external client.

Ways to maintain Security

- **TLS bypass:** The Apple goto "fail bug" was a control-flow error in error handling code that lead to a complete compromise of TLS connections on apple systems.
- **DOS:** A lack of basic error handling can lead to system shutdown.
- This is usually a fairly easy vulnerability for attackers to exploit.
- Other error handling problems could lead to increased usage of CPU or disk in ways that could degrade the system.

Ways to maintain Security

- **Positive Advice**

- Manage exceptions in a centralized manner to avoid duplicated try/catch blocks in the code.
- Ensure that all unexpected behavior is correctly handled inside the application.
- Ensure that error messages displayed to users do not leak critical data, but are still verbose enough to enable the proper user response.
- Ensure that exceptions are logged in a way that gives enough information for support, QA, forensics or incident response teams to understand the problem.
- Carefully test and verify error handling code.

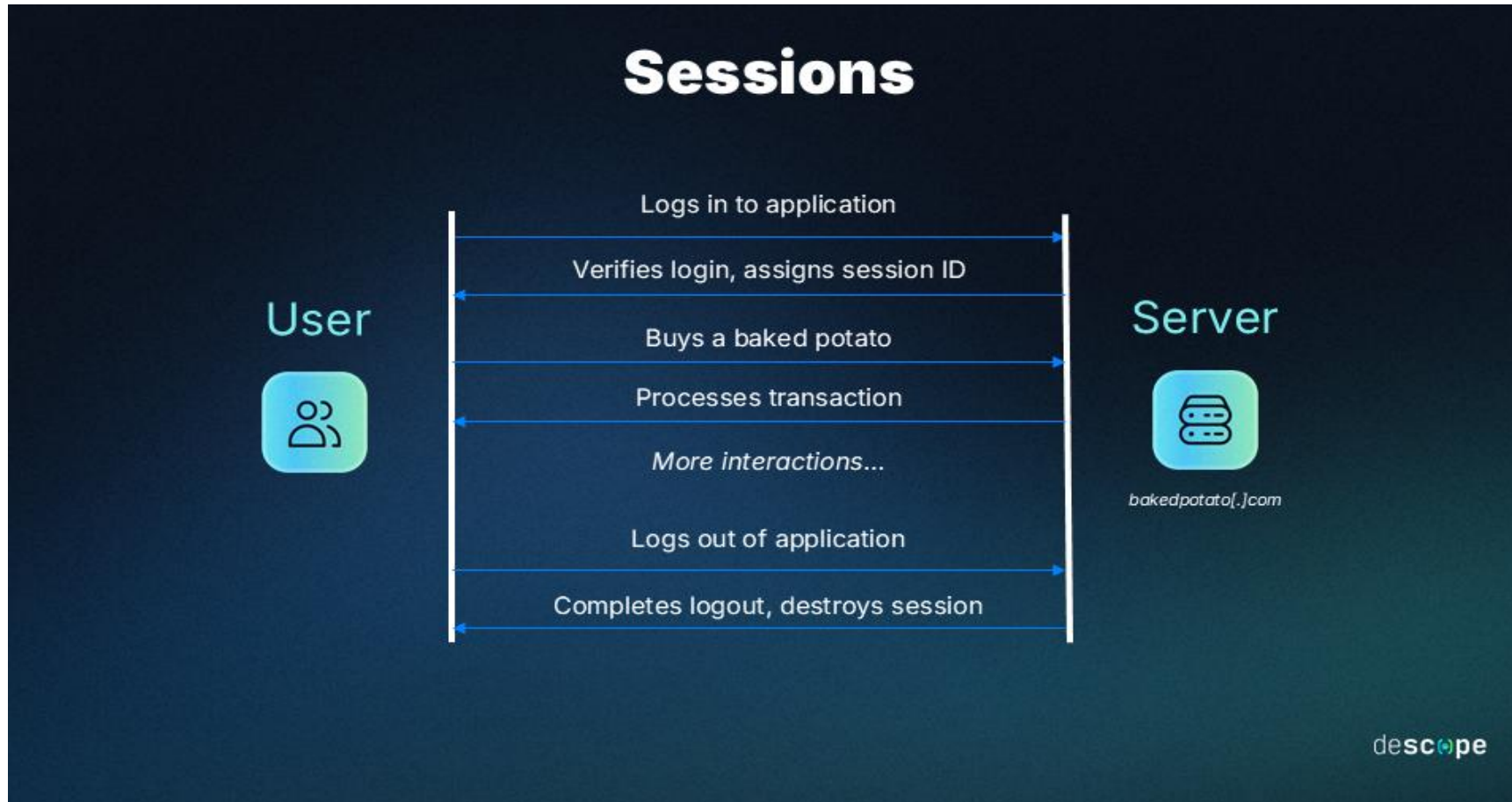
Ways to maintain Security

- **Session management**
- Session management is a cornerstone of creating secure, interactive, and personalized user experiences within web applications.
- As users navigate through web pages, making purchases, reading content, or engaging in various activities, their interactions need to be seamlessly connected and maintained.
- This is challenging given the stateless nature of HTTP – the foundational protocol of the web.
- Session management bridges this gap, enabling applications to remember users' actions and preferences from one page to the next, ensuring that their journey on a website feels cohesive and tailored to their needs.

Ways to maintain Security

- **Understanding session management**
- A session is a finite period of time during which authenticated users are engaging with a system or application.
- A session starts when the user enters the app / website and lasts until the user logs out or is timed out.
- Session management refers to the process of securely handling multiple requests from the same user or client during a single session.
- It involves creating, maintaining, and terminating sessions, ensuring the security of the data exchanged in these sessions, and efficiently managing session data.

Ways to maintain Security



Ways to maintain Security

- **Aspects involved for Session Management:**
- **Creation:** Once a user is authenticated, a session is started by generating a unique session identifier (session ID).
- This session ID is then sent to the client, typically as a cookie or a token, and is used to track subsequent requests from the user.
- **Session Storage:** The server stores the session data, which can include the session ID and any other relevant information about the user's interaction with the web application (ex. user preferences, authentication status, shopping cart items).
- **Maintenance:** With each HTTP request from the user, the session ID is sent to the server.
- The session ID is used to retrieve the corresponding session data, thereby maintaining the interaction between the user and the application.
- If any suspicious or anomalous activity is detected, re-authentication may be required.

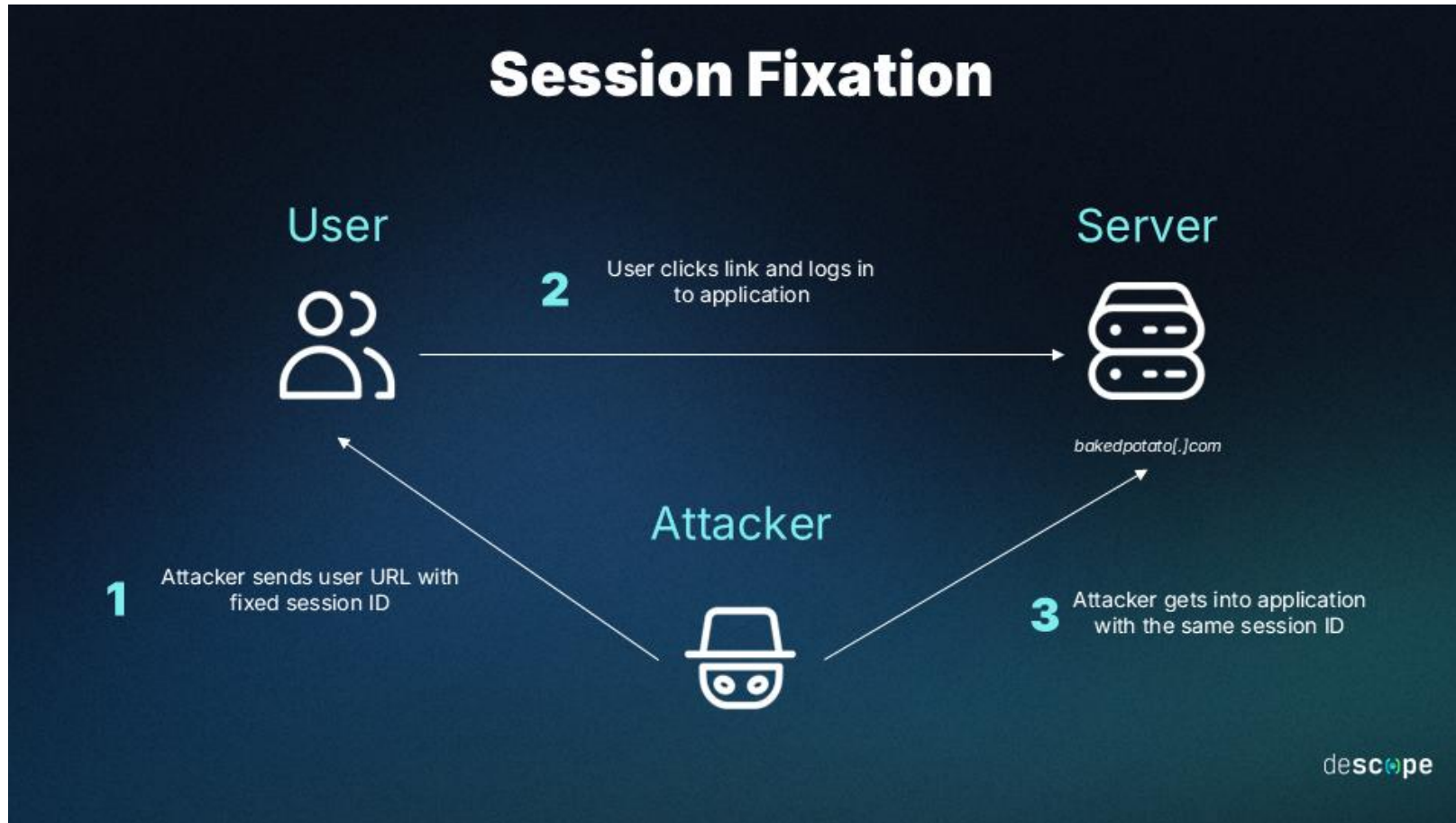
Ways to maintain Security

- **Maintenance:** With each HTTP request from the user, the session ID is sent to the server.
- The session ID is used to retrieve the corresponding session data, thereby maintaining the interaction between the user and the application.
- If any suspicious or anomalous activity is detected, re-authentication may be required.
- **Termination:** Sessions end when users log out or meet other criteria, such as closing their browser or application windows.
- Sessions may be configured to time out after a certain duration, at which point users are logged out or prompted to re-authenticate.

Ways to maintain Security

- **Session management vulnerabilities**
- The stakes of session management couldn't be higher, as specific attacks such as session hijacking and session fixation can lead to cybersecurity breaches.
- **Session hijacking** is a type of man-in-the-middle (MITM) attack in which cybercriminals pose as authenticated users to gain illegitimate access to resources for theft, fraud, extortion, and other illicit ends.
- **Session fixation** is a subset of hijacking in which attackers set session IDs to values they know and can replicate to gain access alongside the user or maintain access after the actual, legitimate access ends.

Ways to maintain Security



Ways to maintain Security

- **Best practices for secure session management**
- **Use secure, random session identifiers:** The session ID, which uniquely identifies user sessions, should be generated using a secure, cryptographically strong random number generator.
- The Open Worldwide Application Security Project (OWASP) session management cheat sheet highlights the importance of session naming conventions.
- An overly simple name makes for easy fingerprinting, which can be leveraged for hijacks and other attacks.
- Likewise, session IDs should be assigned long enough numerical values to prevent brute force attacks leveraging compute to guess (usually shorter) codes.
- OWASP recommends at least 128 bits.

Ways to maintain Security

- **Implement HTTPS for all sessions:** All communication involving session IDs and data should occur over HTTPS, which encrypts the data in transit.
- This prevents attackers from eavesdropping on the communication and stealing session IDs through MITM attacks.
- **Enforce session expiration:** Sessions should have a defined timeout period, after which they are automatically terminated.
- This minimizes the risk of session hijacking by limiting the time an attacker has to exploit an active session.
- Sessions should also be terminated immediately upon user logout or prolonged inactivity.

Ways to maintain Security

- **Regenerate session IDs after login:** Regenerate and issue a new session ID upon every successful user authentication.
- This practice prevents session fixation attacks, where an attacker could pre-establish a session and trick a victim into authenticating under that session ID.

Ways to maintain Security

- **Logging and Auditing**
- **Logging**
- Systematic recording of events, activities, and transactions within an application.
- **Security Applications:**
- **Error Tracking:** Capture exceptions and system failures
- **User Activity:** Record authentication, authorization attempts, and sensitive operations
- **System Health:** Monitor performance metrics and resource usage
- **Third-party Integration:** Track external API calls and responses

Ways to maintain Security

- **Auditing**
- Systematic review and analysis of logs to ensure compliance, detect anomalies, and investigate incidents.
- **Security Applications:**
- **Compliance Verification:** Meet regulatory requirements (GDPR, HIPAA, PCI-DSS)
- **Forensic Analysis:** Investigate security incidents and breaches
- **Behavior Analysis:** Detect unusual patterns and potential threats
- **Access Review:** Validate proper access controls and privilege usage

Ways to maintain Security

- **Critical Security Logging Categories**
- **Authentication & Authorization**
- [SECURITY] Failed login attempt - User: admin, IP: 192.168.1.100, Time: 2024-01-15T10:30:00Z
- [SECURITY] Privilege escalation detected - User: jdoe, New Role: ADMIN, Source: API Key
- [SECURITY] Session hijacking attempt - Invalid session token from IP: 203.0.113.5

Ways to maintain Security

- **Data Access & Modification**
- [DATA] Sensitive record accessed - User: auditor1, Record: Patient-12345, Query: SELECT *
- [DATA] Bulk export detected - User: analyst3, Records: 50,000+, Time: 02:00 AM
- [DATA] Unauthorized schema modification - User: dev_user, Table: payment_info

Ways to maintain Security

- **Input Validation & Injection Attempts**
- [ATTACK] SQL Injection attempt - Input: ' OR '1'='1, Blocked by WAF
- [ATTACK] XSS payload detected - Script: <script>alert()</script>, Sanitized
- [ATTACK] Path traversal - Requested: ../../../../etc/passwd, Denied

Ways to maintain Security

- **Implementation Checklist**
- Implement structured logging format (JSON preferred)
- Ensure log integrity with cryptographic hashing
- Separate security logs from application logs
- Implement role-based access to audit trails
- Set up real-time alerting for critical events
- Define log retention policies (typically 90 days - 7 years)
- Regular vulnerability scanning of logging infrastructure
- Test log restoration procedures
- Document incident response procedures using logs
- Conduct regular log analysis and audit reviews

Ways to maintain Security

- Effective logging and auditing create a **defense-in-depth** strategy by providing:
- **Detection:** Identifying security incidents in real-time
- **Investigation:** Enabling forensic analysis post-incident
- **Deterrence:** Discouraging malicious actions through visibility
- **Compliance:** Demonstrating adherence to security standards
- **Continuous Improvement:** Informing security enhancements through analysis

Security Testing

- **Broken Access control**
- **Broken Access Control** occurs when users can act outside their intended permissions, allowing unauthorized access, modification, or destruction of data.
- It's consistently ranked **#1 in the OWASP Top 10 2025**.
- Common access control vulnerabilities include:
 - Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
 - Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool that modifies API requests.

Security Testing

- Permitting viewing or editing someone else's account by providing its unique identifier (insecure direct object references)
- An accessible API with missing access controls for POST, PUT, and DELETE.
- Elevation of privilege. Acting as a user without being logged in or gaining privileges beyond those expected of the logged in user (e.g. admin access).
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access control token, a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation.
- CORS misconfiguration allows API access from unauthorized or untrusted origins.
- Force browsing (guessing URLs) to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

Security Testing

- **How to prevent**
- Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata.
- Implement access control mechanisms once and reuse them throughout the application.
- Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record.
- Unique application business limit requirements should be enforced by domain models.
- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.

Security Testing

- Log access control failures, alert admins when appropriate (e.g., repeated failures).
- Implement rate limits on API and controller access to minimize the harm from automated attack tooling.
- Stateful session identifiers should be invalidated on the server after logout. Stateless JWT tokens should be short-lived to minimize the window of opportunity for an attacker. For longer-lived JWTs, consider using refresh tokens and following OAuth standards to revoke access.
- Use well-established toolkits or patterns that provide simple, declarative access controls.

Security Testing

- **Cryptographic failures**
- Moving down two positions to #4, this weakness focuses on failures related to the lack of cryptography, insufficiently strong cryptography, leaking of cryptographic keys, and related errors.
- Three of the most common Common Weakness Enumerations (CWEs) in this risk involved the use of a weak pseudo-random number generator:
- CWE-327 Use of a Broken or Risky Cryptographic Algorithm, CWE-331: Insufficient Entropy,
- CWE-1241: Use of Predictable Algorithm in Random Number Generator, and CWE-338 Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG).

Security Testing

- **How to prevent**
- Classify and label data processed, stored, or transmitted by an application. Identify which data is sensitive according to privacy laws, regulatory requirements, or business needs.
- Store your most sensitive keys in a hardware or cloud-based HSM(Hardware Security Module).
- Use well-trusted implementations of cryptographic algorithms whenever possible.
- Don't store sensitive data unnecessarily. Discard it as soon as possible or use PCI DSS compliant tokenization or even truncation. Data that is not retained cannot be stolen.
- Make sure to encrypt all sensitive data at rest.
- Ensure up-to-date and strong standard algorithms, protocols, and keys are in place; use proper key management.

Security Testing

- Encrypt all data in transit with protocols $\geq$ TLS 1.2 only, with forward secrecy (FS) ciphers, drop support for cipher block chaining (CBC) ciphers, support quantum key change algorithms.
- For HTTPS enforce encryption using HTTP Strict Transport Security (HSTS).
- Do not use unencrypted protocols such as FTP, and STARTTLS. Avoid using SMTP for transmitting confidential data.
- Store passwords using strong adaptive and salted hashing functions with a work factor (delay factor), such as Argon2, yescrypt, scrypt or PBKDF2-HMAC-SHA-512.
- Avoid deprecated cryptographic functions, block building methods and padding schemes, such as MD5, SHA1, Cipher Block Chaining Mode (CBC)
- Always use authenticated encryption instead of just encryption.

Security Testing

- **Example attack scenarios**
- **Scenario #1:** A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g., at an insecure wireless network), downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie. The attacker then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data. Instead of the above they could alter all transported data, e.g., the recipient of a money transfer.
- **Scenario #2:** The password database uses unsalted or simple hashes to store everyone's passwords. A file upload flaw allows an attacker to retrieve the password database. All the unsalted hashes can be exposed with a rainbow table of pre-calculated hashes. Hashes generated by simple or fast hash functions may be cracked by GPUs, even if they were salted.

Security Testing

- **Authentication Failures**
- Authentication Failures maintains its position at #7 with a slight name change to more accurately reflect the 36 CWEs in this category. Despite benefits from standardized frameworks, this category has kept its #7 rank from 2021.
- Notable CWEs included are CWE-259 Use of Hard-coded Password, CWE-297: Improper Validation of Certificate with Host Mismatch, CWE-287: Improper Authentication, CWE-384: Session Fixation, and CWE-798 Use of Hard-coded Credentials.

Security Testing

- **How to prevent**
- Where possible, implement and enforce use of multi-factor authentication to prevent automated credential stuffing, brute force, and stolen credential reuse attacks.
- Where possible, encourage and enable the use of password managers, to help users make better choices.
- Do not ship or deploy with any default credentials, particularly for admin users.
- Align password length, complexity, and rotation policies with National Institute of Standards and Technology (NIST) 800-63b's guidelines in section 5.1.1 for Memorized Secrets or other modern, evidence-based password policies.
- Do not force human beings to rotate passwords unless you suspect breach.
- If you suspect breach, force password resets immediately.

Security Testing

- Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks by using the same messages for all outcomes (“Invalid username or password.”).
- Limit or increasingly delay failed login attempts but be careful not to create a denial of service scenario. Log all failures and alert administrators when credential stuffing, brute force, or other attacks are detected or suspected.
- Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login. Session identifiers should not be in the URL, be securely stored in a secure cookie, and invalidated after logout, idle, and absolute timeouts.

Security Testing

- **Example Scenarios**
- **Scenario #1:** Most successful authentication attacks occur due to the continued use of passwords as the sole authentication factor. Once considered best practices, password rotation and complexity requirements encourage users to both reuse passwords and use weak passwords. Organizations are recommended to stop these practices per NIST 800-63 and to enforce use of multi-factor authentication on all important systems.
- **Scenario #2:** Application session timeouts aren't implemented correctly. A user uses a public computer to access an application and instead of selecting "logout," the user simply closes the browser tab and walks away. Another Example for this is, if a Single Sign on (SSO) session can not be closed by a Single Logout (SLO). That is, a single login logs you into, for example, your mail reader, your document system, and your chat system. But logging out happens only to the current system. If an attacker uses the same browser after the victim thinks they have successfully logged out, but with the user still authenticated to some of the applications, then can access the victim's account. The same issue can happen in offices and enterprises when a sensitive application has not been properly exited and a colleague has (temporary) access to the unlocked computer.

Security Testing

- **Software or Data Integrity Failures**
- Software or Data Integrity Failures continues at #8, with a slight, clarifying name change from "Software and Data Integrity Failures".
- This category is focused on the failure to maintain trust boundaries and verify the integrity of software, code, and data artifacts at a lower level than Software Supply Chain Failures.
- This category focuses on making assumptions related to software updates and critical data, without verifying integrity.
- Notable Common Weakness Enumerations (CWEs) include CWE-829: Inclusion of Functionality from Untrusted Control Sphere, CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes, and CWE-502: Deserialization of Untrusted Data.

Security Testing

- **How to prevent**
- Use digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered.
- Ensure libraries and dependencies, such as npm or Maven, are only consuming trusted repositories. If you have a higher risk profile, consider hosting an internal known-good repository that's vetted.
- Ensure that there is a review process for code and configuration changes to minimize the chance that malicious code or configuration could be introduced into your software pipeline.
- Ensure that your CI/CD(Continuous integration/Continuous Deployment) pipeline has proper segregation, configuration, and access control to ensure the integrity of the code flowing through the build and deploy processes.
- Ensure that unsigned or unencrypted serialized data is not received from untrusted clients and subsequently used without some form of integrity check or digital signature to detect tampering or replay of the serialized data.

Security Testing

- **Example attack scenarios**
- **Scenario #1 Inclusion of Web Functionality from an Untrusted Source:** A company uses an external service provider to provide support functionality. For convenience, it has a DNS mapping for myCompany.SupportProvider.com to support.myCompany.com. This means that all cookies, including authentication cookies, set on the myCompany.com domain will now be sent to the support provider. Anyone with access to the support provider's infrastructure can steal the cookies of all of your users that have visited support.myCompany.com and perform a session hijacking attack.
- **Scenario #2 Update without signing:** Many home routers, set-top boxes, device firmware, and others do not verify updates via signed firmware. Unsigned firmware is a growing target for attackers and is expected to only get worse. This is a major concern as many times there is no mechanism to remediate other than to fix in a future version and wait for previous versions to age out.

Security Testing

- **Scenario #3 Use of Package from an Untrusted Source:** A developer has trouble finding the updated version of a package they are looking for, so they download it not from the regular, trusted package manager, but from a website online. The package is not signed, and thus there is no opportunity to ensure integrity. The package includes malicious code.
- **Scenario #4 Insecure Deserialization:** A React application calls a set of Spring Boot microservices. Being functional programmers, they tried to ensure that their code is immutable. The solution they came up with is serializing the user state and passing it back and forth with each request. An attacker notices the "rO0" Java object signature (in base64) and uses the Java Deserialization Scanner to gain remote code execution on the application server.

Security Testing

- **Security Logging and Monitoring Failures**
- Security Logging & Alerting Failures retains its position at #9.
- This category has a slight name change to emphasize the alerting function needed to induce action on relevant logging events.
- This category will always be underrepresented in the data, and for the third time voted into a position in the list from the community survey participants.
- This category is incredibly difficult to test for, and has minimal representation in the CVE/CVSS data (only 723 CVEs); but can be very impactful for visibility and incident alerting and forensics.
- This category includes issues with properly handling output encoding to log files (CWE-117), inserting sensitive data into log files (CWE-532), and insufficient logging (CWE-778).

Security Testing

- **How to prevent**
- Ensure all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
- Ensure that every part of your app that contains a security control is logged, whether it succeeds or fails.
- Ensure that logs are generated in a format that log management solutions can easily consume.
- Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- Ensure all transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.

Security Testing

- Ensure all transactions that throw an error are rolled back and started over. Always fail closed.
- If your application or its users behave suspiciously, issue an alert. Create guidance for your developers on this topic so they can code against this or buy a system for this.
- DevSecOps and security teams should establish effective monitoring and alerting use cases including playbooks such that suspicious activities are detected and responded to quickly by the Security Operations Center (SOC) team.
- Add ‘honeytokens’ as traps for attackers into your application e.g. into the database, data, as real and/or technical user identity. As they are not used in normal business, any access generates logging data that can be alerted with nearly no false positives.

Security Testing

- **Example attack scenarios**
- **Scenario #1:** A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.
- **Scenario #2:** A major Indian airline had a data breach involving more than ten years' worth of personal data of millions of passengers, including passport and credit card data. The data breach occurred at a third-party cloud hosting provider, who notified the airline of the breach after some time.

Security Testing

- **Scenario #3:** A major European airline suffered a GDPR reportable breach. The breach was reportedly caused by payment application security vulnerabilities exploited by attackers, who harvested more than 400,000 customer payment records. The airline was fined 20 million pounds as a result by the privacy regulator.

THANK YOU